

Educação Profissional Paulista

Técnico em
**Ciência de
Dados**

Estrutura de Controle de Fluxo

Loops “while”

AULA 1

Código da aula: [DADOS]ANO1C2B1S4A1

Exposição



Objetivo da Aula

Conhecer as técnicas de Repetições em *Python*



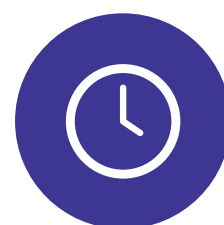
Competências da Unidade (Técnicas e Socioemocionais)

- Ser proficiente em linguagens de programação para manipular e analisar grandes conjuntos de dados;
- Colaborar efetivamente com outros profissionais, como cientistas de dados e engenheiros de dados; trabalhar em equipes multifuncionais colaborando com colegas, gestores e clientes para alcançar objetivos comuns.



Recursos Didáticos

- Acesso ao laboratório de informática e/ou internet;
- Software Anaconda/Jupyter Notebook instalado.



Duração da Aula

50 minutos

O que já sabemos

Estrutura de Controle de Fluxo

- ✓ Aplicamos estruturas condicionais;
- ✓ Relembramos a condicional *IF*, *ELIF* e *ELSE*;
- ✓ Juntamos o conceito com entrada e saída de dados, além de operações.

Motivação:

Fazer à mão ou programar?

O que você faria para exibir na tela os número de 1 a 5 (um em cada linha)?

O código abaixo atenderia esse objetivo?

```
In [1]: print(1)
        print(2)
        print(3)
        print(4)
        print(5)
```

```
1
2
3
4
5
```

Reprodução - Jupyter Notebook



Momento
de **reflexão**

© Pexels

Motivação:

Fazer à mão ou programar?

A partir do exemplo apresentado no slide anterior, responda:

- 1.** Como exibir os números de 1 a 100, um em cada linha?
- 2.** Quem vai fazer à mão?
- 3.** O que precisa para não ter muito esforço?



Momento
de **reflexão**

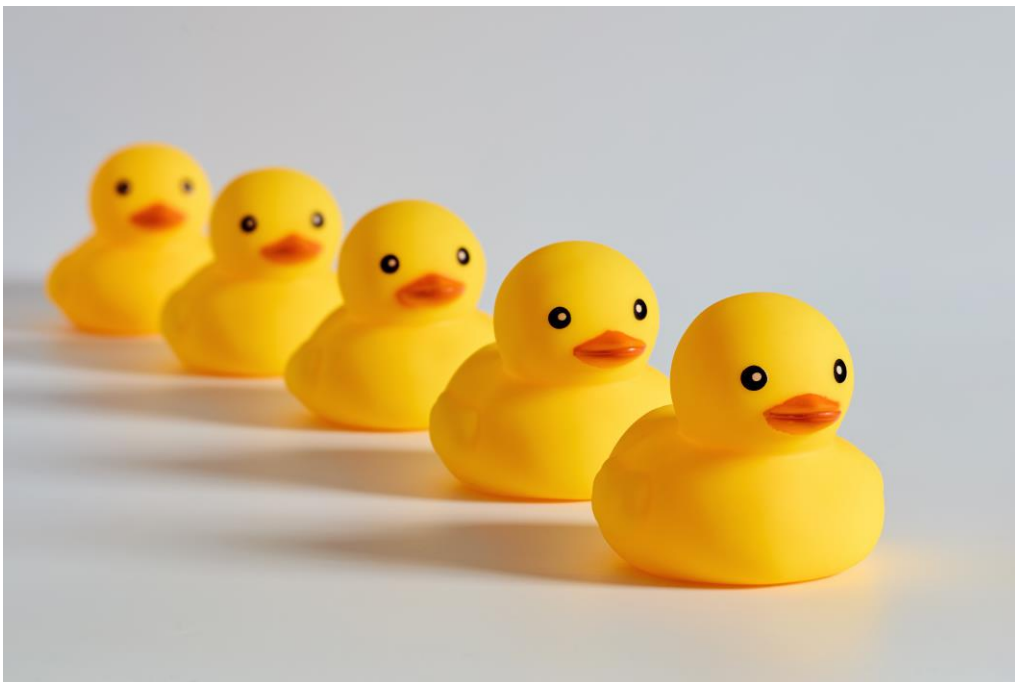
© Pexels

Exposição

Repetições

Em *Python*, utilizamos as repetições para executar um bloco de código várias vezes. Isso pode ser feito até que uma condição seja atendida por um número predefinido de iterações.

Podemos continuar repetindo esse bloco até que algo específico aconteça ou, simplesmente, decidir quantas vezes queremos que ele rode.



© Getty Images



Tome nota

Existem duas principais formas de implementar repetições em *Python*: o loop "*for*" e o loop "*while*".

Exposição

Repetições: na prática

Vamos voltar ao problema apresentado no início da aula e tentar responder à questão: **Como exibir os números de 1 a 100, um em cada linha?**

Veja que, nesse exemplo, o código começa com X acumulando 1.

Após mostrar o resultado na tela, adicionamos mais um, mas ainda de forma manual.

Agora, que tal pensarmos juntos: existe um jeito de continuar somando 1 automaticamente até chegarmos a 100?

Como vocês acreditam que podemos automatizar isso em *Python*?"

```
In [2]: x = 1  
print(x)  
x = x + 1  
print(x)  
x = x + 1  
print(x)  
x = x + 1  
print(x)  
x = x + 1  
print(x)
```

```
1  
2  
3  
4  
5
```

Reprodução – Jupyter Notebook

Repetições com o loop *while*

Estrutura básica do *while*

Usamos *while* (em inglês é enquanto) para executar um bloco de código repetidamente, enquanto uma determinada condição for verdadeira. A condição é verificada antes de cada iteração.

Sua sintaxe é:

```
while condição:  
    # bloco de código a ser executado enquanto a condição for verdadeira
```

Reprodução - Jupyter Notebook



Na prática

Imagine que você está numa corrida em que só continua correndo enquanto tiver fôlego. No *Python*, o *while* é parecido: ele repete um trecho do código enquanto a condição que definimos for verdadeira, checando isso antes de cada volta.

Exposição

Repetições com o loop *while*: na prática

Vamos retomar aquela questão sobre como podemos escrever 5 números de maneira automática? Deem uma olhada neste código abaixo.

O que vocês notam nele? Vamos analisar juntos e entender como ele funciona?

Na linha 1 começamos a variável `x = 1`.

A linha 3 é a execução, se for verdadeira.

```
1 x=1
2 while x<5:
3     print(x)
4     x=x+1
5
```

A linha 2 significa: enquanto `x` for menor ou igual a 5, faça...

A linha 4 soma 1 ao `x` (contadores). A execução volta para a linha 2 e faz tudo de novo até que a condição "`x` é menor ou igual a 5" seja falsa.

1
2
3
4
5

Elaborado especialmente para o curso com Jupyter Notebook.

Repetições com *while*

O que esse código faz?
Quando ele sai do loop? Quando ele para?

```
x = 1
while x > 5:
    print(x)
    x = x + 1
```

Reprodução - Jupyter Notebook



Atenção!

Não executem esse código! Cuidado com o loop infinito

Repetições com *while*

Loop infinito

É importante ter cuidado para evitar loops infinitos, nos quais a condição nunca se torna falsa, fazendo com que o loop nunca seja interrompido.



Atenção!

Cuidado com o loop infinito! Para evitar isso, é preciso garantir que a condição eventualmente se torne falsa, ou utilizar mecanismos como *break* ou *continue* para controlar o fluxo do loop.

Exposição

Repetições com o loop *while*

Interrompendo a execução

Para interromper o *while*, podemos usar o `break` (do inglês, parar)

Quando o comando "break" é encontrado dentro do loop, o fluxo de execução é imediatamente transferido para fora do loop, ignorando qualquer código restante que deveria ser executado dentro do loop.



© Getty Images



Dica

O uso do "break" é útil quando se deseja interromper um loop com base em alguma condição específica, mesmo que a condição original para a execução do loop ainda não tenha sido satisfeita.

(MENEZES, 2019)

Repetições com loop *while*

Neste exemplo, o loop "*while*" é iniciado com a condição "*True*", o que significa que ele será executado indefinidamente. No entanto, dentro do loop, é verificado se o valor do contador é maior ou igual a 5.

```
: # Exemplo de loop "while" com break
```

```
contador = 0
```

```
while True:
```

```
    print(contador)
```

```
    contador += 1
```

```
    if contador >= 5:
```

```
        break
```

```
print("Loop interrompido!")
```

```
0
```

```
1
```

```
2
```

```
3
```

```
4
```

```
Loop interrompido!
```

Se a condição for satisfeita, o comando "break" é acionado, interrompendo o loop e fazendo com que o programa imprima "Loop interrompido!" na saída.

Reprodução - Jupyter Notebook

Repetições com loop *while*

Estruturas aninhadas

Podemos ter, também, estruturas aninhadas em que eu tenho um *while* dentro do outro.

```
linha = 1

while linha <= 5:
    coluna = 1

    while coluna <= linha:
        print("*", end=" ")
        coluna += 1

    print()
    linha += 1
```

```
*
* *
* * *
* * * *
* * * * *
```

Reprodução – Jupyter Notebook



Vamos
fazer uma
atividade

Fazer à mão ou programar?

Com base em tudo que exploramos nesta aula, chegou a hora de vocês mostrarem o que aprenderam! Vamos ao desafio:

Essa atividade
deverá ser enviada
para o AVA
(Ambiente Virtual de
Aprendizagem)



10min



Em grupo

- 1** Criem um código em *Python* que exiba os números de 1 a 100, cada um em sua própria linha.
- 2** Pensem nas repetições e na automação que discutimos.
- 3** Como vocês vão estruturar esse loop para que ele faça exatamente o que estamos pedindo?



© Getty Images

O que nós
**aprendemos
hoje?**

Hoje desenvolvemos:

- 1** O conceito de estrutura de repetição;
- 2** A compreensão da sintaxe do loop *while*;
- 3** O conhecimento de estratégias para evitar que o código entre em loop infinito.

Saiba mais

Quer saber mais sobre estruturas de repetição?
Acesse o curso “Python para Data Science: primeiros passos.”

ALURA. Python para Data Science: primeiros passos. Disponível em:
<https://cursos.alura.com.br/course/python-data-science-primeiros-passos/task/122395>. Acesso em:
18 jan. 2024.

E já conhece o laço while?

ALURA. Python para Data Science: primeiros passos. Disponível em:
<https://cursos.alura.com.br/course/python-data-science-primeiros-passos/task/122396>. Acesso em:
18 jan. 2024.

Referências da aula

MENEZES, N. N. C. **Introdução à programação com Python**: algoritmos e lógica de programação para iniciantes. São Paulo: Novatec, 2019.

Identidade visual: imagens © Getty Images

**Educação
Profissional
Paulista**

Técnico em
**Ciência de
Dados**